

Forecast one series with ARIMA

20 minutes of ideas in four chapters. 40 minutes in one notebook.

CCIS faculty workshop | Oct 2, 2026 | 3 PM to 4 PM

Slides, notebooks, and web explorer: github.com/ahleksu/arima-workshop-ccis

The hour

Minute	What happens
0 to 2	Chapter 1. Intro
2 to 8	Chapter 2. What is ARIMA
8 to 14	Chapter 3. Requirements
14 to 20	Chapter 4. Fitting, then the handoff
20 to 23	Open notebook 03 in Colab
23 to 55	ARIMA in Python. Three steps in notebook 03
55 to 60	Recap

Chapter 1. Intro

What comes next?

- A time series is a list of values in time order
- ARIMA learns how the past links to the next value, and projects that link forward
- It is also the **baseline** that a newer model must beat

Today in four steps: **what ARIMA is, what it needs, how you fit it, and how to run it in Python.**

Name one series from your own work.

Chapter 2. What is ARIMA

AR plus I plus MA

Part	What it does	Dial
AR	Predicts the next value from the last p values	p
I	Differences the series d times so that it holds steady	d
MA	Corrects the forecast with the last q errors	q

The model decides how much of each part to use.

Two kinds of memory

AR(p): memory of **values**

$$y_t = c + \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} + \varepsilon_t$$

MA(q): memory of **shocks**

$$y_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}$$

I: difference first, $y'_t = y_t - y_{t-1}$. ARIMA adds the AR part and the MA part to y'_t .

Chapter 3. Requirements

Requirement 1: a steady series

Plot first. Then check that the series is **stationary**, which means that its rules do not change over time:

- the mean stays constant
- the spread stays constant
- the link between a value and the value k steps earlier depends only on k

ARIMA learns **one** set of coefficients. That only works if the pattern holds in the future. A trend breaks it.

Requirement 2: choose the orders p , d , and q . The next chapter shows how.

Differencing and the ADF test

Difference: replace each value by its change, $y_t - y_{t-1}$. A rising line becomes a flat line.

ADF test

- Null hypothesis: the series is **non-stationary** (unit root)
- **Small** p-value (below 0.05): the series looks stationary

Our weekly data (notebook 03): level $p = 0.51$, first difference p near 0. So $d = 1$.

Difference as little as possible. Each extra difference adds noise.

Chapter 4. Fitting

Reading the fingerprints

Process	ACF	PACF
AR(p)	Fades	Stops after lag p
MA(q)	Stops after lag q	Fades
Both	Fades	Fades
White noise	No spikes	No spikes

The plot that stops names the order.

The blue band is about $\pm 1.96 / \sqrt{n}$. One spike in twenty crosses it by chance.

Score the candidates

1. Choose **d**: difference until stationary, and no more
2. Read the ACF and PACF for candidate **p** and **q**
3. Fit the candidates and compare **AIC** and **BIC** (lower is better)
4. Within about 2 points, take the **simpler** model

The software finds the coefficients by **maximum likelihood**: the values that make your data most probable.

Our result (notebook 03)

- Weekly demand, 182 training weeks
- ADF says $d = 1$
- Lowest AIC: ARIMA(2,1,1). Lowest BIC: ARIMA(0,1,1)
- Five models are within 2 AIC points, so the **parsimony rule** picks **ARIMA(0,1,1)**

You will build this yourself in step 2.

Handoff

For the next 40 minutes:

1. Open notebook `03` with the Colab button on the Hands-on page
2. Run the cells **top to bottom** (Shift + Enter)
3. Read the text above each cell before you run it
4. Answer the check question at the end of each step
5. Raise a hand when a cell fails

ARIMA in Python. Step 1

Prepare the series

10 minutes. Run sections 1 and 2.

Plot the weekly demand, hold out the last 26 weeks, and test the level and the first difference with the ADF test.

Check question

What is the ADF p-value of the level, and of the first difference?

ARIMA in Python. Step 2

Choose and fit

12 minutes. Run sections 3, 4, and 5.

Read the ACF and PACF of the differenced series. Compare candidate orders with AIC and BIC. Keep the simplest order that scores well, and fit it.

Check question

Which order does the parsimony rule pick, and which orders have the lowest AIC and the lowest BIC?

ARIMA in Python. Step 3

Check and forecast

10 minutes. Run sections 7 and 8. Skip section 6.

- The **residuals** (what the model missed) must look like white noise
- **Ljung-Box**: a **large** p-value is good here. This is the opposite of ADF.
- **Hold-out**: forecast the last 26 weeks and compare

Check question

Does the model pass the Ljung-Box test, and how far off is the forecast?

Recap

- Plot first, test second
- Difference as little as possible
- The plot that stops names the order
- Fit, then diagnose, then decide
- A forecast needs a baseline and a hold-out test

After today: notebooks 01 and 02 (background), 04 (seasonal terms and outside drivers), 05 (validation), 06 (your own data).

Reviewing a forecasting chapter: 8 questions

1. Is the data source, frequency, and period stated?
2. Were breaks and outliers found and explained?
3. How was stationarity decided, and which tests?
4. How were orders chosen? Is the grid stated?
5. Do the residual checks pass?
6. Is there a baseline, and does the model beat it?
7. Is the evaluation out of sample, from many origins?
8. Are intervals reported, with their coverage?

Your next hour

1. Open **notebook 06**. Run it once on the sample.
2. Change `DATA_PATH` and four settings. Run it on your series.
3. Read the data checklist output before you trust anything.
4. Copy the methods paragraph and edit it.

Privacy: keep sensitive data off Colab. Use the local setup script.

When ARIMA fits, and when it does not

Good fit

- One series, regular spacing
- 50 to a few thousand points
- Patterns you can describe: trend, season, memory
- You need intervals and a model you can explain

Poor fit

- Many related series, or huge feature sets
- Strong non-linear or regime-switching behavior
- Very irregular or event-driven data
- Very long seasons (365 with daily data)

Where to go next

- *Forecasting: Principles and Practice* (3rd edition), free online: otexts.com/fpp3
- statsmodels time series docs: statsmodels.org/stable/tsa.html
- `pmdarima` docs: alkaline-ml.com/pmdarima
- Extensions: ARIMAX with lagged drivers, state-space models, ETS, hierarchical forecasting, probabilistic metrics

Everything from today: github.com/ahleksu/arima-workshop-ccis

Free and open. Code MIT. Content CC BY 4.0.

Questions

Come back to the series you wrote on the board at the start.